JavaScript Task Manager

Concepts & Mistakes Review

A study guide based on the JavaScript code and debugging discussion from this chat.

1. Main Concepts Used

Concept	What it means
DOM Selection	Using methods such as getElementById() and querySelectorAll() to access HTML elements from JavaScript.
Event Listeners	Using addEventListener() to run code when the user clicks a button.
Dynamic DOM Elements	Elements created later with innerHTML do not exist when earlier querySelectorAll() code runs.
Event Delegation	Listening on a parent such as document and using closest() to detect dynamically-created buttons.
data-* Attributes	Using data-index and data-type to store information on each task button.
Arrays	Moving task objects between toDoTasksArray, progressTasksArray, and completeTasksArray.
localStorage	Saving arrays as JSON strings and restoring them with JSON.parse().
innerHTML	Generating task cards and buttons dynamically by assigning HTML strings to container elements.
Array splice()	Removing a task from its original array after moving it to another array.
Array push()	Adding the selected task object to its new array.

2. The Most Important Mistake: Dynamic Buttons

Your task buttons are created inside functions such as showToDoTasks(), showProgressTasks(), and showCompleteTasks(). The cards are inserted using innerHTML. The click listeners, however, were attached later in the script using querySelectorAll(). Because the buttons are dynamically generated, the NodeList can be empty when the listener code runs.

Problematic pattern

```
document.querySelectorAll(".start-btn").forEach(btn => {
btn.addEventListener("click", () => { ... });
});
```

Better solution: Event Delegation

```
document.addEventListener("click", function (e) {
const btn = e.target.closest(".to-do-btn, .start-btn, .complete-btn");
if (!btn) return;

const index = Number(btn.dataset.index);
const fromName = btn.dataset.type;
```

```
if (btn.classList.contains("to-do-btn")) {
addToDoTasksArray(index, fromName);
} else if (btn.classList.contains("start-btn")) {
addToProgressTasksArray(index, fromName);
} else if (btn.classList.contains("complete-btn")) {
addToCompleteTasksArray(index, fromName);
}
});
```

Why this works: document already exists when the JavaScript runs. Events from the dynamically-created buttons bubble up to document, where closest() finds the actual button that was clicked.

3. Mistake: Wrong Function for the Start Button

The Start button in the To Do column should move a task from To Do → Progress. But the original code called addToDoTasksArray(), which attempts to move the task back into the To Do array.

```
Incorrect logic

if (fromName === 'toDo') {
addToDoTasksArray(index, fromName);
}

Correct logic

if (fromName === 'toDo') {
addToProgressTasksArray(index, fromName);
}
```

4. Mistake: Saving the Wrong Array to localStorage

Inside addToDoTasksArray(), the task is pushed into toDoTasksArray, but the original code saved completeTasksArray to localStorage. This causes the stored To Do data to become inconsistent.

```
Incorrect

localStorage.setItem("completeTasksArray", JSON.stringify(completeTasksArray));

Correct

localStorage.setItem("toDoTasksArray", JSON.stringify(toDoTasksArray));
```

5. Task Movement Logic

Button	From	To	Function
Start	To Do	Progress	addToProgressTasksArray()
Complete	To Do	Complete	addToCompleteTasksArray()
To Do	Progress	To Do	addToDoTasksArray()
Complete	Progress	Complete	addToCompleteTasksArray()
To Do	Complete	To Do	addToDoTasksArray()
Start	Complete	Progress	addToProgressTasksArray()

6. Understanding data-index and data-type

Each button stores the task's array position and its source array. For example:

```
Start
```

JavaScript can read these values with btn.dataset.index and btn.dataset.type. Since dataset values are strings, Number(btn.dataset.index) converts the index to a number.

7. How Your Move Functions Work

The general pattern is: get the source array → get the task by index → push the task into the destination array → remove it from the source array → save the changed arrays → redraw the UI.

```
const source = getArrayByName(fromName);
const card = source.array[cardIndex];
destinationArray.push(card);
source.array.splice(cardIndex, 1);
localStorage.setItem(...);
showAllTasks();
```

8. Other Issues Not Yet Implemented

The Update and Delete buttons are present in the generated cards, but updateTask() and deleteTask() are empty. Therefore those buttons cannot perform an action yet.

You also use repeated IDs such as id="start-btn", id="complete-btn", and id="to-do-btn" for multiple task cards. IDs are intended to be unique. For repeated buttons, classes and data attributes are the better choice.

9. Debugging Checklist

• Open the browser Console and check for JavaScript errors.

• Check whether querySelectorAll() finds the buttons at the moment the listener is attached.

• If elements are created with innerHTML, consider event delegation.

• Check that each button calls the function matching its intended action.

• Check the source and destination arrays before and after moving a task.

• Check that the correct array is saved under the correct localStorage key.

• Use console.log(index, fromName) to verify data-index and data-type.

• After changing arrays, call showAllTasks() to redraw the UI.

10. Quick Summary

The key lesson: When you generate buttons dynamically with innerHTML, attaching event listeners to those buttons before/after the wrong point in the lifecycle can make them appear but not respond. Event delegation is a simple and reliable solution. Also make sure the button action, array movement function, and localStorage key all match the intended task transition.